IIITA Timetabling System

A thesis submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

By:-

Eshant

Rahul Roy

Lakavath Peer Singh

Enrollment No.

IIT2023198

IIT2023196

IIT2023197

Under the Supervision of

Dr. Gaurav Srivastava

to the

Department of Information Technology

Indian Institute of Information Technology,
Allahabad

07/05/2026

भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad
An Institute of National Importance by Act of Parliament
Deoghat Jhalwa, Prayagraj 211015 (U.P.) India
Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CERTIFICATE

It is certified that the work contained in the thesis titled "IIITA Timetabling System" by Eshant, Rahul Roy, Lakavath Peer Singh has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Dr. Gaurav Srivastava
Department of Information Technology
IIIT Allahabad

भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad
An Institute of National Importance by Act of Parliament
Deoghat Jhalwa, Prayagraj 211015 (U.P.) India
Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CANDIDATE DECLARATION

We, Eshant (IIT2023198), Rahul Roy (IIT2023196), and Lakavath Peer Singh (IIT2023197), hereby certify that the thesis titled "IIITA Timetabling System" has been submitted by our in partial fulfillment of the requirements for the Degree of Bachelor of Technology in the Department of Information Technology at the Indian Institute of Information Technology, Allahabad.

We acknowledge that plagiarism includes:

1. Reproducing another person's work (fully or partially) or ideas and presenting them as our own.
2. Copying or paraphrasing another person's work without appropriate attribution.
3. Engaging in literary theft by reproducing a unique literary construct without crediting its source.

We affirm that due credit has been given to all original authors and sources through proper citations for words, ideas, diagrams, graphics, computer programs, experiments, results, and websites that are not our own contributions. Direct quotations have been appropriately marked, and sources have been properly referenced. Furthermore, we declare that no portion of our work is plagiarized. In the event of a plagiarism claim, we accept full responsibility for the contents of this thesis. We acknowledge that our supervisor may not be able to verify the originality of every aspect of our work.

Place: **IIITA** Date: **07/05/2026**

Eshant (IIT2023198)

Rahul Roy (IIT2023196)

Lakavath Peer Singh (IIT2023197)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, ALLAHABAD

IIITA Timetabling System

ABSTRACT

University timetabling is a highly complex, NP-hard constraint satisfaction problem that significantly impacts academic operations and resource utilization. In modern educational institutions, managing course schedules, professor assignments, room allocations, and student groups requires a robust, dynamic, and integrated system. This thesis presents the design, architecture, and implementation of the "IIITA Timetabling System," a comprehensive platform developed specifically to address these challenges at the Indian Institute of Information Technology, Allahabad.

The proposed system digitizes the entire timetable lifecycle. It introduces a heuristic-driven Excel parsing engine capable of ingesting raw, unstructured timetable spreadsheets, extracting critical metadata (such as Subject Codes, Faculty Names, and Room Numbers), and automatically resolving cell merges and varying time formats. This data is securely stored and synchronized with a cloud-based PostgreSQL backend powered by Supabase.

A central contribution of this thesis is the implementation of an automated timetable generator powered by a (1+1) Evolution Strategy (ES) algorithm. The solver models each timetable as a chromosome of Gene decisions (day, time-slot, room) and evaluates candidate schedules against a fitness function comprising ten hard constraints (room non-overlap, professor non-overlap, student group non-overlap, break/lunch boundary enforcement, lab-room assignment, elective basket synchronization, WMC-section isolation, home-room binding, the 2+1 lecture format, and time boundary compliance) and three soft constraints (student schedule gaps, room utilization efficiency, and professor same-half-day avoidance). The algorithm

employs Rechenberg's 1/5th success rule for step-size (σ) adaptation, a stagnation restart mechanism, and supports warm-starting from existing seed timetables. Cross-semester constraint awareness is achieved through locked sessions from published timetables.

To complement the automated solver, the system provides an interactive drag-and-drop Timetable Editor. This component enables administrators to manually adjust solver-generated schedules, visualize conflicts in real time with colour-coded highlights, and invoke a Large Neighbourhood Search (LNS) auto-fix routine that targets clashing sessions. The editor features a Master View that aggregates all published timetable slots across semesters, enabling cross-timetable conflict detection.

To serve various stakeholders, the system provides role-specific, dynamic viewing interfaces. Students benefit from a personalized "My Timetable" view with conflict resolution and intelligent merging of lecture and practical sessions. Faculty members receive dedicated schedules, while administrators can access a campus-wide "Master Occupancy" grid and a "Free Room Finder" utility. The system also includes an advanced clash detection algorithm that identifies room overlaps, professor scheduling conflicts, and section-level clashes in real time.

Built using React 19, TypeScript, and Tailwind CSS, the application offers a responsive, modern user experience. It features robust integration with the Google Sheets API for professional formatted data export, alongside localized PDF generation capabilities. This thesis details the theoretical foundation of the timetabling problem, the modern technology stack, the modular software architecture, the evolutionary algorithm and its constraint system, and the interactive editing tools implemented to achieve a scalable, real-time campus timetabling solution.

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Dr. Gaurav Srivastava, for his continuous support, guidance, and encouragement throughout this research and development process. His insightful comments and technical suggestions have been invaluable in shaping this work.

We are also grateful to the faculty members of the Department of Information Technology for their support during our studies. Special thanks to our colleagues and friends who provided a stimulating environment for learning and development.

Finally, we would like to thank our families for their unconditional love, support, and patience throughout our academic journey.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The administration of academic institutions involves numerous complex logistical challenges, among which the generation, management, and distribution of timetables stand out as one of the most resource-intensive tasks. The University Timetabling Problem (UTP) is a well-known constraint satisfaction problem that has garnered significant attention from both the operations research and computer science communities. In modern educational environments, the necessity for a dynamic, robust, and accessible system to manage these schedules has become paramount. This chapter provides a comprehensive introduction to the IIITA Timetabling System, outlining the background, motivation, problem statement, objectives, and the overall organization of the thesis.

1.1 Background and Motivation

Academic timetabling involves the assignment of events—such as lectures, tutorials, and practical laboratory sessions—to a limited number of resources, including time slots, rooms, and teaching faculty. This assignment must satisfy a myriad of constraints. These constraints are generally categorized into two distinct types: hard constraints, which cannot be violated under any circumstances (such as double-booking a professor or scheduling a class in an

occupied room), and soft constraints, which are desirable but not strictly necessary (such as minimizing the number of gaps between classes for students or avoiding late-evening classes).

Traditionally, timetabling at the Indian Institute of Information Technology, Allahabad (IIITA) and many similar institutions has been largely a manual or semi-automated process. Academic coordinators rely on spreadsheet software, such as Microsoft Excel, to draft and refine schedules. While Excel provides a flexible canvas for data entry, it lacks inherent semantic understanding of the data it holds. Consequently, detecting scheduling conflicts—such as overlapping classes for a specific student section or a professor being assigned to two different locations simultaneously—relies heavily on human vigilance. This manual verification is prone to error, especially given the scale of a modern university with hundreds of courses, dozens of rooms, and thousands of students.

Furthermore, once a timetable is generated in a static format like a PDF or a spreadsheet, distributing updates becomes a significant challenge. Changes in room allocations or faculty assignments necessitate the redistribution of the entire document, leading to version control issues where students or faculty might refer to outdated schedules. The static nature of these documents also means they cannot be easily queried. For instance, a student wishing to find a free room for group study, or an administrator wanting to visualize the overall campus occupancy at a specific hour, cannot easily extract this information from a flat Excel file.

The motivation for the IIITA Timetabling System stems directly from these inefficiencies. There is a pressing need for a centralized, cloud-based platform that can act as a single source of truth for all academic scheduling data. By transitioning from static files to a dynamic database-driven architecture, it becomes possible to offer customized, role-based views of the schedule, perform real-time clash detection, automate the generation of conflict-free schedules using evolutionary algorithms, and facilitate seamless updates.

1.2 Problem Statement

The core problem addressed by this thesis is the inefficiency, error-proneness, and static nature of traditional, spreadsheet-based university timetable management, combined with the lack of automated schedule generation. Specifically, the existing process suffers from the following critical limitations:

- **Data Silos and Unstructured Formats:** Timetables are often maintained in unstructured Excel formats containing complex cell merges, inconsistent naming conventions, and implicit relationships that are difficult for machines to parse.
- **Lack of Automated Conflict Resolution:** Identifying clashes (room double-booking, professor overlaps, or section schedule conflicts) is a manual process, leading to inevitable logistical errors that disrupt academic operations.
- **Absence of Automated Schedule Generation:** No system exists at the institutional level to automatically compute a conflict-free timetable from scratch given a set of subjects, professors, rooms, and student groups. The entire process remains manual and iterative.
- **Absence of Role-Specific Visibility:** A singular master timetable is difficult to interpret for individual stakeholders. Students must manually filter through extensive documents to find their specific classes, while faculty struggle to isolate their teaching commitments.
- **Poor Resource Utilization Tracking:** Administrators have no centralized mechanism to quickly identify available rooms or assess the overall occupancy rate of campus infrastructure at any given time.

- **Static Distribution:** Once published, schedules cannot be easily updated dynamically, resulting in communication gaps and reliance on outdated physical or digital copies.

Therefore, the objective is to design and implement a comprehensive software solution that can bridge the gap between human-readable spreadsheet formats and machine-readable relational data, automate the generation of conflict-free schedules, provide interactive editing tools, and enable dynamic, intelligent, and accessible timetable management.

1.3 Objectives of the Study

To address the problems outlined above, this project sets forth a series of specific, measurable objectives. The primary goal is to develop a full-stack web application that serves as a centralized hub for all scheduling activities. The specific objectives are as follows:

1. **Heuristic Data Ingestion:** To design and implement a sophisticated parsing engine capable of reading complex, unstructured Excel timetable files. This engine must intelligently extract metadata (subjects, faculty, rooms), resolve cell merges (common in multi-hour laboratory sessions), and normalize varying time formats into a structured relational schema.

2. **Real-Time Clash Detection:** To develop robust algorithms capable of analyzing the structured timetable data to instantly identify and highlight scheduling conflicts across multiple dimensions, including rooms, professors, and student sections.

3. **Role-Based Dynamic Visualization:** To create specialized user interfaces tailored to the needs of different stakeholders. This includes:

- A personalized "My Timetable" view for students, featuring subject filtering and elective selection.
- A dedicated "Professor View" for teaching faculty.
- A "Master Occupancy View" and "Free Room Finder" for campus administrators to optimize infrastructure usage.

4. **Export and Reporting Mechanisms:** To integrate seamless export functionality, allowing users to generate high-quality localized PDFs of their schedules, and enabling administrators to export complex, heavily formatted schedule reports directly to Google Sheets using the Google Sheets API.

5. **Automated Timetable Generation:** To implement a fully functional automated timetable generator using a (1+1) Evolution Strategy (ES) algorithm. This solver must evaluate candidate schedules against a comprehensive fitness function encompassing ten hard constraints and three soft constraints, employ adaptive step-size control via the 1/5th success rule, support warm-starting from existing seed timetables, and respect cross-semester constraints through locked published sessions.

6. **Interactive Timetable Editor:** To develop a drag-and-drop editing interface that allows administrators to manually refine solver-generated or imported timetables. This editor must provide real-time conflict visualization, inline editing of room and professor assignments, and an automated Large Neighbourhood Search (LNS) auto-fix routine to resolve remaining clashes.

7. **Multi-Step Generator Wizard:** To build a guided configuration workflow that enables administrators to select semester clusters, map home rooms, assign professors to subjects via an interactive matrix, and launch the solver with configurable seed and

locked timetable options.

1.4 Scope and Limitations

The scope of this project encompasses the design, development, and deployment of the IIITA Timetabling System frontend using React and TypeScript, the backend infrastructure using Supabase (PostgreSQL), and the automated timetable solver using a (1+1) Evolution Strategy implemented entirely in client-side TypeScript. The system is specifically tailored to handle the complexities of the timetable format used at IIIT Allahabad, though the architectural principles are extensible to other institutions.

It is important to clearly define the limitations of the current system:

- **Heuristic Parser Boundaries:** The Excel parser relies on specific heuristic rules (e.g., header row detection, column mapping). Significant deviations from the standard institutional template may require manual intervention or updates to the parsing logic.
- **Single-Threaded Solver:** The (1+1)-ES solver executes on the browser's main thread using batched asynchronous scheduling (via `setTimeout`). While sufficient for current problem sizes, institutions with significantly larger datasets may require a migration to Web Workers or a server-side solver for improved performance.
- **Heuristic Optimality:** The Evolution Strategy is a meta-heuristic. While it reliably produces feasible (zero hard-violation) schedules, the solutions are not guaranteed to be globally optimal with respect to soft constraints. The quality of solutions depends on the number of generations and the specific mutation landscape.

1.5 Organization of the Thesis

The remainder of this thesis is structured as follows to provide a logical progression from theory to implementation and finally to evaluation:

Chapter 2: Literature Review explores the existing body of work related to university timetabling. It discusses the theoretical underpinnings of the University Timetabling Problem (UTP) as an NP-hard constraint satisfaction problem and reviews various algorithmic approaches and existing software solutions in the academic and commercial spheres.

Chapter 3: Technologies and Frameworks provides a detailed technical overview of the modern web development stack chosen for this project. It delves into the rationale behind selecting React 19, TypeScript, Tailwind CSS, Vite, and the Supabase Backend-as-a-Service (BaaS) platform, explaining how these tools contribute to the system's performance and maintainability.

Chapter 4: System Architecture and Design dissects the structural blueprint of the application. It presents the relational database schema, detailing the entities and their complex relationships. Furthermore, it explains the data flow from raw Excel ingestion through the parsing pipeline to the cloud database, the modular architecture of the solver engine, and finally to the client-side React components.

Chapter 5: Implementation and Results is the core technical chapter of the thesis. It provides an in-depth look at the implementation of critical system modules, including the heuristic Excel parser, the multi-dimensional clash detection algorithms, the (1+1)-ES solver algorithm with its ten hard constraints and three soft constraints, the interactive drag-and-drop Timetable Editor with LNS-based auto-fix, and the multi-step Generator Wizard. This chapter also showcases the final user interfaces and discusses the integration of external APIs

such as Google Sheets.

Chapter 6: Conclusion and Future Work summarizes the key contributions of the project, reflects on the challenges overcome during development, and outlines promising avenues for future research and enhancement, particularly focusing on parallelized solvers and advanced meta-heuristic hybridization.

Chapter 2

Literature Review

The challenge of creating conflict-free, optimal schedules for academic institutions has been a subject of extensive research for decades. This chapter reviews the existing literature surrounding the University Timetabling Problem (UTP), categorizes the various algorithmic approaches proposed by researchers, examines the evolution of software architectures used in timetable management systems, and identifies the gaps in existing solutions that this project seeks to address.

2.1 The University Timetabling Problem (UTP)

Academic timetabling is formally defined as the process of assigning a set of events (such as lectures, exams, or meetings) to a set of resources (such as time slots, rooms, and faculty members) subject to a multitude of constraints [**schaerf1999survey**]. The complexity of this task is heavily dependent on the size of the institution, the flexibility of the curriculum, and the rigidity of the constraints.

2.1.1 Constraint Satisfaction and Complexity

In computational complexity theory, the University Timetabling Problem is generally classified as NP-hard [**even1975complexity**]. This implies that as the number of variables

(classes, rooms, professors) increases, the time required to find an optimal solution grows exponentially, making brute-force search algorithms entirely unfeasible for real-world applications.

Constraints within the UTP are universally divided into two categories:

- **Hard Constraints:** These are non-negotiable conditions that must be satisfied for a timetable to be considered feasible. Examples include:
  - A room cannot host more than one class simultaneously.
  - A professor cannot teach two different classes at the same time.
  - A student section cannot be scheduled for two different lectures simultaneously.
  - The seating capacity of an assigned room must meet or exceed the number of students enrolled in the class.
- **Soft Constraints:** These represent preferences that enhance the quality of the timetable but are not strictly necessary for its execution. A penalty score is usually assigned when soft constraints are violated, and the goal of optimization algorithms is to minimize this score. Examples include:
  - Minimizing the number of free periods (gaps) in a student's daily schedule.
  - Ensuring faculty members do not have heavily fragmented schedules across the week.
  - Assigning classes to rooms preferred by the instructor.
  - Grouping specific core subjects together sequentially.

2.2 Algorithmic Approaches to Timetabling

Over the years, researchers have proposed a vast array of techniques to solve the UTP, ranging from classical mathematical modeling to modern meta-heuristic algorithms.

2.2.1 Exact Methods

Early approaches focused on exact mathematical methods, primarily Integer Linear Programming (ILP) and Graph Coloring. In a Graph Coloring model, events are represented as nodes, and edges are drawn between events that conflict (e.g., they share the same student group or professor). The goal is to "color" the nodes—where each color represents a distinct time slot—using the minimum number of colors such that no two connected nodes share the same color [**de1985graph**]. While theoretically sound, exact methods struggle significantly with the high dimensionality of real-world timetabling problems, often failing to find solutions within acceptable computation times when soft constraints are introduced.

2.2.2 Heuristic and Meta-Heuristic Algorithms

Due to the computational limitations of exact methods, the focus of the research community shifted heavily towards heuristic and meta-heuristic approaches in the late 1990s and 2000s. These methods do not guarantee finding the absolute optimal solution but are highly effective at finding "good enough" (feasible and high-quality) solutions within reasonable timeframes.

- **Genetic Algorithms (GA):** Inspired by natural evolution, GAs represent potential timetables as "chromosomes." Through processes of selection, crossover (combining parts of two schedules), and mutation (randomly altering a scheduled event), populations of timetables evolve over generations to minimize constraint violations [**burke1996genetic**].

- **Simulated Annealing (SA) and Tabu Search:** These local search techniques iteratively make small changes to a complete schedule. Tabu search uses a memory structure (a "tabu list") to prevent the algorithm from revisiting recently evaluated schedules, thereby avoiding getting trapped in local optima [**glover1997tabu**]. Simulated Annealing mimics the metallurgical process of annealing, occasionally accepting "worse" solutions early in the process to explore a broader search space before gradually "cooling" down to focus on strict optimization [**kirkpatrick1983optimization**].

2.3 Evolution of Timetable Management Systems

While academic research has heavily focused on the mathematical generation of schedules, the practical software engineering aspects of timetable management—how the data is ingested, visualized, and distributed—have evolved significantly alongside broader trends in software development.

2.3.1 Legacy Systems: Spreadsheets and Desktop Applications

Historically, timetables were drafted manually on large whiteboards or physical grids. With the advent of personal computing, spreadsheet software like Microsoft Excel became the de-facto standard for academic coordinators. Excel offers a highly flexible, human-readable grid interface. However, as noted by numerous educational technologists, spreadsheets are fundamentally static. They lack the relational data models necessary to automatically enforce referential integrity or detect complex logical clashes without the use of fragile, custom-built macros (VBA scripts) [**panko1998what**]. Furthermore, distributing an Excel file means distributing a static snapshot in time, leading to severe communication breakdowns when schedules inevitably change mid-semester.

In response to the limitations of spreadsheets, institutional desktop applications were developed in the late 1990s and early 2000s using technologies like Java Swing or .NET WinForms. These systems introduced relational databases (like MySQL or Oracle) to store scheduling data, offering significantly better integrity than flat files. However, being desktop-bound, they still suffered from distribution issues. Stakeholders (students and faculty) typically could not interact directly with the system, relying instead on printed reports or static HTML exports generated by administrators.

2.3.2 Modern Web-Based Architectures

The shift towards cloud computing and modern web frameworks has revolutionized how institutions handle internal logistics. Modern systems utilize full-stack web architectures, allowing centralized data storage with ubiquitous access via web browsers.

The adoption of modern frontend frameworks like React, Angular, or Vue.js has enabled the creation of Highly Interactive Single Page Applications (SPAs). Unlike older web technologies that required full page reloads for every interaction, SPAs provide dynamic, fluid user experiences. This is particularly crucial for timetabling interfaces, where users need to rapidly switch between different views (e.g., moving from a "Room View" to a "Professor View") or apply complex filters (e.g., viewing only specific student sections) without latency.

Furthermore, the rise of Backend-as-a-Service (BaaS) platforms, such as Firebase or Supabase, has accelerated the development of such tools. By abstracting the complexities of server maintenance, database provisioning, and authentication, developers can focus more on the domain-specific logic—such as conflict detection algorithms or Excel parsing heuristics.

2.4 Identification of Gaps in Existing Solutions

Despite the proliferation of generic scheduling tools and complex academic research on constraint solving, a noticeable gap remains for many mid-to-large scale academic institutions, particularly in developing nations.

1. **The Ingestion Bottleneck:** Commercial systems often require data to be entered manually through their proprietary interfaces or imported using highly rigid CSV templates. Institutions that rely on complex, idiosyncratic Excel formats (with merged cells, color-coding, and embedded textual notes) find it extremely difficult to migrate their existing workflows into these rigid systems. There is a critical need for heuristic parsers capable of understanding "human-formatted" spreadsheets.

2. **Lack of Integrated Occupancy Tracking:** Many timetabling systems focus solely on the student/faculty schedule output. They lack dedicated tools for campus administrators to view real-time infrastructure utilization. Features like a "Free Room Finder" or a macroscopic "Master Occupancy Grid" are rarely integrated seamlessly with the core scheduling engine.

3. **Disconnect Between Generation and Management:** Academic tools that generate perfect schedules using Genetic Algorithms often feature poor, academic user interfaces that are unusable by administrative staff. Conversely, polished commercial management systems often treat the schedule as a static entity to be displayed, rather than a dynamic puzzle to be solved.

The literature thus establishes a clear trajectory: from purely mathematical algorithms that fail to account for human factors, to localized spreadsheet software that lacks systemic integration, to modern cloud-based systems that attempt to unify these domains.

The system developed in this thesis builds upon this foundation, adopting the "Modern

Table 2.1: Comparison of Timetabling Methodologies

Feature	**Spreadsheets (Legacy)**	**Pure Algorithms**	**Modern Cloud Systems**
Conflict Detection	Manual & Error-Prone	Fully Automated	Real-time Automated
Data Accessibility	Offline / Localized	Varies (often CLI based)	Cloud-synchronized (Web/Mobile)
Flexibility	High (free-form data entry)	Low (strict mathematical constraints)	High (heuristics + constraints)
Integration	None	Low	High (APIs, Database links)
User Interface	Basic Grid	Academic / Non-existent	Role-based Dashboards

Cloud System" paradigm while specifically integrating a heuristic parser to bridge the gap with legacy spreadsheet data, laying a clean architectural foundation for future automated constraint-solving integration.

Chapter 3

Technologies and Frameworks

The development of the IIITA Timetabling System required the selection of a modern, robust, and highly scalable technology stack. This chapter details the technical foundation upon which the system is built. It provides an in-depth analysis of the frontend frameworks, styling methodologies, backend services, external Application Programming Interfaces (APIs), and the solver engine architecture integrated into the project.

Table 3.1: Summary of the Technology Stack

Layer	**Technology**	**Primary Function**
Frontend Framework	React 19	Component-based UI rendering and virtual DOM management
Programming Language	TypeScript	Static typing for interfaces and error reduction
Build Tool	Vite	Fast hot-module replacement and optimized asset bundling
Styling	Tailwind CSS	Utility-first styling for rapid, responsive UI development
Database / Backend	PostgreSQL (Supabase)	Relational data storage with referential integrity
Authentication	Supabase Auth	Secure user session management
External APIs	Google Sheets API	Formatted data export to external workbooks

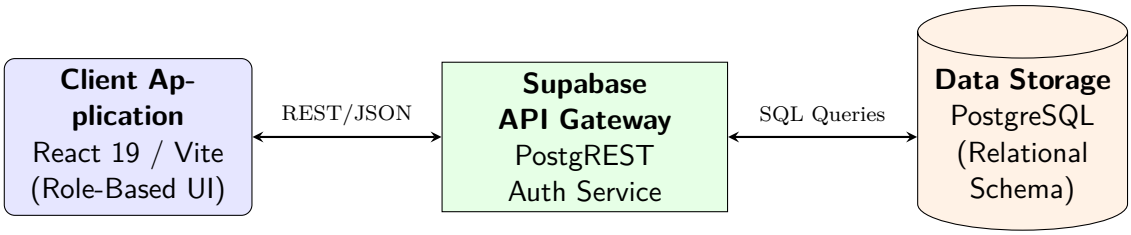

FIGURE 3.1: High-Level System Architecture and Component Interaction

3.1 Frontend Architecture: React 19 and TypeScript

The user interface (UI) represents the critical interaction layer for all stakeholders—students, faculty, and administrators. A timetable application is inherently data-heavy, requiring the rendering of complex, dense grids that must update instantly based on user input (such as filtering by specific sections or switching between professor views). To achieve this fluidity, a Component-Based Architecture was adopted.

3.1.1 React 19

React, maintained by Meta, is a premier JavaScript library for building user interfaces. It was selected as the core frontend framework for this project due to its declarative nature and the efficiency of the Virtual Document Object Model (Virtual DOM).

In a traditional web application, frequent updates to complex tables (such as a 5-day by 10-hour timetable grid) directly manipulate the browser's Document Object Model (DOM), an operation that is computationally expensive and slow. React mitigates this by maintaining a lightweight, in-memory representation of the UI (the Virtual DOM). When state changes—for instance, when a student toggles an elective subject off—React computes the exact difference (the "diff") between the previous Virtual DOM and the new one. It then batches these changes and updates only the specific DOM nodes that changed, ensuring high-performance rendering even with dense tabular data.

The project utilizes React 19, taking advantage of modern paradigms such as Hooks (‘useState‘, ‘useEffect‘, ‘useMemo‘). The ‘useMemo‘ hook, in particular, proved vital for performance optimization. It allows the application to memoize the results of expensive computational tasks—such as parsing the timetable array to detect multidimensional clashes—ensuring these calculations only run when the underlying timetable data changes, rather than on every single render cycle.

3.1.2 TypeScript

While JavaScript is the native language of the web, it is dynamically typed, which can lead to significant runtime errors in complex applications. To ensure code reliability and maintainability, the entire frontend was developed using TypeScript.

TypeScript is a syntactic superset of JavaScript that adds static typing. In the context of the timetable management system, TypeScript allowed for the rigid definition of data interfaces. For example, the ‘ParsedSlot‘ interface strictly defines that every parsed timetable entry must possess a string for the subject code, an integer for the day of the week, and specific enumerated types for the slot nature (e.g., ’Lecture’, ’Tutorial’, ’Practical’).

```
// Example TypeScript Interface
interface ParsedSlot {
    id: string;
    day: string;
    startTime: string;
    endTime: string;
    subjectCode: string;
    type: 'Lecture' | 'Tutorial' | 'Practical';
```

```
    facultyName: string;
    roomName: string;
    section: string;
}
```

By enforcing these data contracts at compile-time, TypeScript eradicated an entire class of common errors related to undefined object properties or type mismatches, which is exceptionally beneficial when dealing with the unpredictable outputs of the heuristic Excel parser.

3.2 Build Tooling: Vite

Historically, large React applications relied on Webpack as a build tool and module bundler. However, Webpack's build times and hot-module-replacement (HMR) speeds scale linearly with the size of the application, leading to sluggish developer experiences in extensive projects.

This project utilizes **Vite** (French for "fast") as its build tool. Vite fundamentally shifts the development paradigm by leveraging native ES Modules in the browser. Rather than crawling, building, and bundling the entire application before the server can start, Vite pre-optimizes dependencies using esbuild (written in Go) and serves source code directly to the browser on demand. This results in near-instantaneous server starts and lightning-fast HMR, regardless of the application's complexity. For a project heavily reliant on rapid UI iteration—such as tweaking the dynamic time column generation algorithm for the viewer components—Vite significantly accelerated the development lifecycle.

3.3 Styling: Tailwind CSS

To achieve a professional, modern, and responsive aesthetic without the overhead of maintaining vast, complex CSS stylesheets, the project employs **Tailwind CSS**.

Unlike traditional semantic CSS frameworks (like Bootstrap) that provide pre-designed components (e.g., '.btn-primary'), Tailwind is a utility-first CSS framework. It provides low-level utility classes that map directly to underlying CSS properties (e.g., 'flex', 'pt-4', 'text-center', 'rotate-90').

This approach proved highly advantageous for rendering the timetable grid:

- **Colocation of Concerns:** Styling is applied directly within the React JSX components. This ensures that the structural logic (HTML), behavioral logic (JavaScript), and presentation logic (CSS) for a component—such as an individual timetable cell—reside in exactly the same place, drastically improving code readability.
- **Dynamic Styling:** Utility classes can be easily concatenated conditionally using JavaScript. For instance, the system dynamically assigns text colors based on the slot type: 'text-orange-600' for practicals, 'text-green-800' for tutorials, and black for standard lectures.
- **Performance:** The project utilizes the Vite plugin for Tailwind CSS v4, which scans the source code and generates a minimal CSS file containing only the utility classes actually used in the application, ensuring minimal network payloads in production.

3.4 Backend Infrastructure: Supabase

Rather than developing a custom backend server using Node.js or Python from scratch, the project leverages **Supabase**, an open-source alternative to Firebase. Supabase provides a

suite of Backend-as-a-Service (BaaS) tools built atop a dedicated PostgreSQL database.

3.4.1 PostgreSQL

At the core of Supabase is PostgreSQL, an advanced, enterprise-grade open-source relational database. A relational model is essential for a timetabling system, which fundamentally consists of interconnected entities: subjects, professors, rooms, and student groups. PostgreSQL enforces referential integrity through foreign key constraints, ensuring, for example, that a timetable slot cannot be assigned to a room that does not exist in the master rooms table.

3.4.2 Supabase Client and Row Level Security (RLS)

Supabase provides an auto-generated, RESTful Data API derived directly from the PostgreSQL schema. The React frontend interacts with the database using the '@supabase/supabase-js' client library. This allows for rapid data fetching and mutation directly from the client without the need to manually construct API endpoints.

To ensure data security when communicating directly from the client, Supabase utilizes PostgreSQL's native Row Level Security (RLS). While not strictly enforced in the current developmental phase, the architecture supports strict RLS policies. This ensures that while unauthenticated students can query the `timetable_slots` to view their schedules, only authenticated administrative users hold the necessary permissions to `INSERT` or `DELETE` records via the Excel parsing module.

3.4.3 Authentication

Authentication is managed natively by Supabase Auth. The system currently supports standard Email/Password authentication. The authentication state is managed globally within

the React application using an ‘AuthContext‘ provider, which wraps the main application shell and dynamically routes users to either the login form or the main dashboard based on their session status.

3.5 External Integrations

3.5.1 Excel Parsing (SheetJS / xlsx)

To bridge the gap between traditional administrative workflows and the new digital platform, the system must process raw ‘.xlsx‘ files. This is achieved using the ‘xlsx‘ (SheetJS) library. SheetJS is a highly robust JavaScript library capable of reading complex spreadsheet binaries directly in the browser. It parses the workbook into a JSON array of arrays, which the custom heuristic engine within the ‘TimetableImporter‘ component then processes to extract semantic meaning.

3.5.2 Google Sheets API integration

While the primary viewing experience is web-based, administrators often require the ability to manipulate data further or share raw schedules externally. To support this, the project integrates with the **Google Sheets API v4**.

Using an implementation centered around the ‘useGoogleSheets‘ custom React hook, the application utilizes the Google Identity Services (GSI) OAuth 2.0 client. Upon administrator authorization, the system constructs a complex JSON payload that not only creates a new Google Spreadsheet containing the schedule data but also transmits detailed batch update requests. These updates apply professional formatting directly via the API, including freezing header rows, applying pink background colors to headers, enforcing solid borders,

and setting bold typography. This level of integration ensures that the exported data is immediately presentable and formatted according to institutional standards.

3.5.3 PDF Generation (html-to-image and jsPDF)

For localized, immediate offline access, every viewer component supports PDF export. This is implemented via a two-step client-side process: 1. **'html-to-image'**: This library captures a snapshot of the specific React component (the DOM node representing the timetable grid) and converts it into a high-fidelity base64 PNG image. A critical technical implementation detail involves temporarily overriding sticky CSS headers during the capture process to ensure the entire scrolling grid is captured accurately. 2. **'jsPDF'**: The generated PNG image is then injected into a PDF document canvas utilizing the 'jsPDF' library. The resulting document is automatically downloaded to the user's local machine, providing a quick, formatted snapshot of the current view.

3.6 Solver Engine: Pure TypeScript Architecture

A key architectural decision for the automated timetable generator was to implement the entire solver engine in pure TypeScript, running directly in the browser. This eliminates the need for an external optimizer service, a dedicated server-side process, or any third-party optimization library. The solver module resides in the `src/solver/` directory and is organized into the following submodules:

- `types.ts` — Defines the core data structures: `ClassSession` (a single schedulable event), `Gene` (the allele encoding day, startBucket, and roomIndex), `Solution` (an array of Genes representing a complete timetable), `SolverConfig` (hyperparameters such as weights and thresholds), and `FitnessResult` (the evaluated quality of a solution).

- **`constants.ts`** — Encodes the institutional time model: 8 one-hour slots per day, break boundaries after slots 2 and 4, slot-to-time mappings, and the definition of synced elective baskets (e.g., HSMC, MDM).
- **`dataPrep.ts`** — Transforms UI-level configuration (subjects with L-T-P counts, student groups, professor assignments) into the flat `ClassSession[]` array consumed by the solver. Implements the 2+1 lecture expansion rule and builds locked sessions from published timetables.
- **`constraints.ts`** — Evaluates a candidate solution against all ten hard constraints and three soft constraints, returning a comprehensive `FitnessResult` with a per-constraint violation breakdown.
- **`mutations.ts`** — Provides five mutation operators (day reassignment, Gaussian time shift with break snapping, type-aware room reassignment, swap, and full relocate) and the initial solution generator with round-robin day balancing and cross-basket anti-clash placement.
- **`solver.ts`** — Implements the main (1+1)-ES optimization loop with elitist selection, σ-adaptation via the 1/5th success rule, stagnation restart, and asynchronous batched execution.
- **`localSearch.ts`** — Provides the Large Neighbourhood Search (LNS) engine used by the Timetable Editor for targeted clash resolution.
- **`solverLog.ts`** — Generates comprehensive diagnostic log files documenting the solver's input, performance, fitness, detailed violation report, and day-by-day schedule.

This modular architecture ensures that each concern (data preparation, constraint evaluation, mutation, and optimization control) is cleanly separated, facilitating independent

testing and future extension.

Chapter 4

System Architecture and Design

The design of the IIITA Timetabling System is predicated on the principles of modularity, data integrity, and role-based accessibility. This chapter provides a comprehensive breakdown of the system architecture, detailing the overall application topology, the relational database schema, the administrative data ingestion pipeline, the solver module architecture, and the routing logic that drives the various stakeholder interfaces.

4.1 Application Topology

The system follows a classic decoupled client-server architecture, modified for the modern web via a Backend-as-a-Service model.

- **Client Tier (React SPA):** The frontend is a Single Page Application (SPA) responsible for all user interactions, data formatting, client-side clash detection, and PDF generation. It communicates asynchronously with the backend via secure REST API calls over HTTPS.
- **Backend Tier (Supabase):** Acting as the serverless backend, Supabase provides an auto-generated API gateway, handles user authentication, and interfaces directly with the underlying database.

- **Data Tier (PostgreSQL):** The relational database stores all normalized institutional data, including users, entities (rooms, subjects, professors), and the core timetable slots.

4.2 Database Schema Design

A well-structured relational schema is paramount for a scheduling system to maintain data integrity and support complex queries. The database is heavily normalized to prevent data duplication and anomalies. The core schema comprises the following primary entities and relationships:

4.2.1 Core Entities

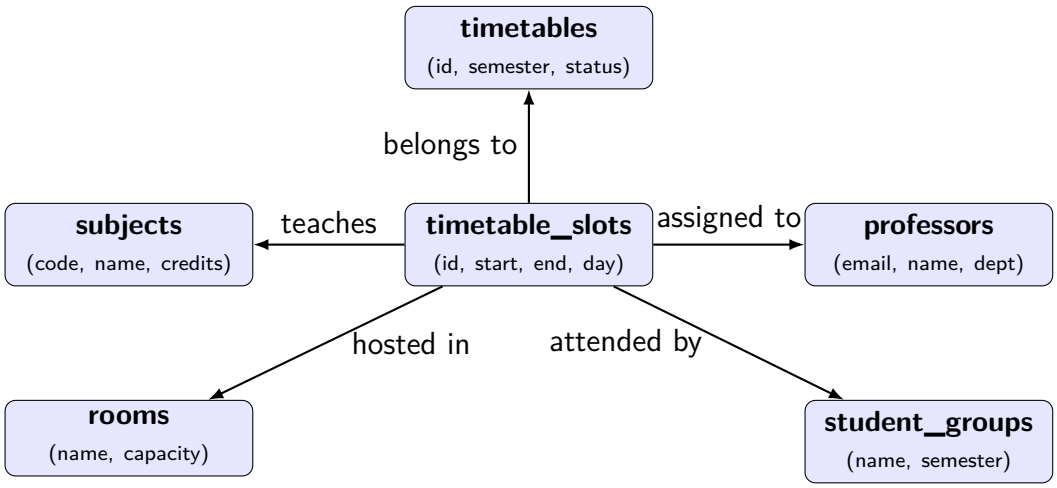

FIGURE 4.1: Simplified Entity-Relationship Diagram of the Core Schema

- **'timetables':** Represents a specific iteration of a schedule (e.g., "B.Tech Sem 3 - Monsoon 2024"). It acts as the parent container for all scheduled slots. Attributes include the academic year, semester, publication status, and institutional break times (lunch start/end).

- **'subjects':** Contains metadata for all offered courses. Attributes include the unique subject code, name, credit distribution (Lectures, Tutorials, Practicals), subject type (Core, Elective, Minor), and elective grouping data.
- **'professors':** Stores faculty information, uniquely identified by email, alongside their departmental affiliation.
- **'rooms':** Catalogs the physical infrastructure of the campus. Attributes include the unique room name (e.g., "CC-3-1001"), seating capacity, and room type (e.g., Lecture Hall, Computer Lab).
- **`student_groups`:** Represents a specific cohort of students, defined by a unique combination of group name (e.g., "Sec A") and semester.

4.2.2 The Central Hub: `timetable_slots`

The `timetable_slots` table is the central junction of the schema. It represents a specific class occurrence in time and space. It acts as a massive join table, linking a specific `timetable_id` to a `subject_id`, `professor_id`, `room_id`, and `student_group_id`.

Crucially, it also defines the temporal coordinates of the event, as detailed in Table 4.1.

4.2.3 Generator and Solver Support Entities

To support automated timetable generation, the schema includes mapping tables that define the constraints and requirements for specific semesters. These tables are actively used by the Generator Wizard and the solver engine:

- **`semester_clusters`:** Defines active academic clusters (e.g., Batch 2021, Semester 5, IT Department). The generator UI uses these clusters to scope the scheduling problem.

Table 4.1: Schema Specification for `timetable_slots`

Column Name	Data Type	Description & Constraints
`id`	`UUID`	Primary Key, auto-generated.
`timetable_id`	`UUID`	Foreign Key referencing `timetables`.
`subject_id`	`UUID`	Foreign Key referencing `subjects`.
`professor_id`	`UUID`	Foreign Key referencing `professors`.
`room_id`	`UUID`	Foreign Key referencing `rooms`.
`day_of_week`	`Integer`	$1 = \text{Monday}, \ldots, 5 = \text{Friday}$.
`start_time`	`VarChar`	24-hour format (e.g., "09:00").
`end_time`	`VarChar`	24-hour format (e.g., "11:00").
`slot_type`	`VarChar`	Enum: 'Lecture', 'Tutorial', 'Practical'.

- **`cluster_requirements`:** A many-to-many join table linking a `semester_cluster` to the `subjects` that must be scheduled for that cohort. Includes `estimated_enrollment` for room utilization optimization.
- **`professor_expertise`:** A mapping table linking `professors` to `subjects` they are qualified to teach, along with a preference level. This is used by the auto-fill feature in the Generator Wizard to intelligently suggest professor assignments.

4.3 The Data Ingestion Pipeline

A major architectural feature of the system is its ability to ingest unstructured data. The transition from an administrator's raw Excel file to normalized database records follows a strict pipeline managed by the `TimetableImporter` module:

1. **Binary Parsing:** The user uploads an '.xlsx' file. The 'xlsx' library reads the binary stream and converts the active spreadsheet into a two-dimensional JSON array.
2. **Header Synchronization:** The heuristic engine scans the top rows to identify the

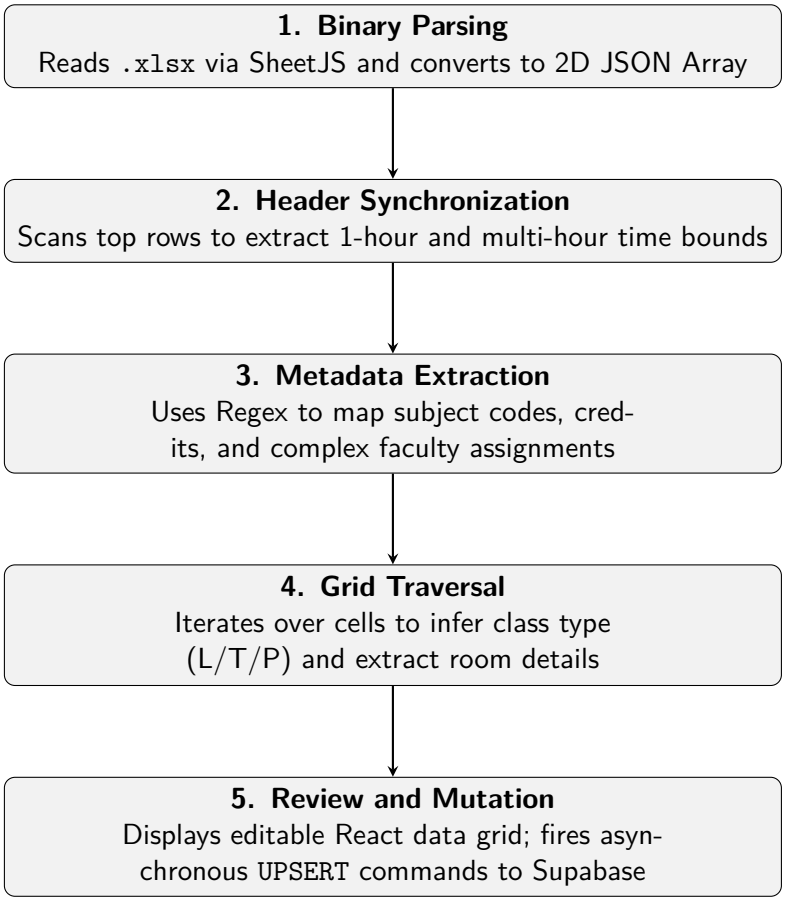

FIGURE 4.2: Data Ingestion Pipeline for Excel Timetables

time boundary headers. It extracts standard 1-hour blocks (e.g., 9:00-10:00) and multi-hour blocks.

3. **Metadata Extraction:** The engine locates the "Course Code / Faculties" section of the spreadsheet. It applies Regular Expressions (Regex) to map subject codes to full names and extracts the L-T-P-S credit structures. It also parses complex faculty assignment strings (e.g., identifying that "Prof. X" teaches "Sec A & B" while "Prof. Y" teaches "Sec C").

4. **Grid Traversal and Extraction:** The engine iterates over the main timetable grid. For each cell, it:

   - Identifies the current day from the leftmost column.
   - Identifies the time bounds from the column header.
   - Extracts the subject code, infers the class type (L/T/P) based on cell coloring or text markers, and extracts room and section assignments.

5. **Review and Mutation:** The extracted raw data is presented to the administrator in an editable React data grid. Upon confirmation, the application fires a sequence of asynchronous `UPSERT` commands to Supabase, ensuring that all necessary entities (professors, rooms, subjects) exist before performing a bulk `INSERT` into the `timetable_slots` table.

4.4 View Routing and State Management

The application does not utilize a traditional URL-based router (like React Router). Instead, it employs state-based view switching managed by the root ‘App.tsx‘ component.

A central state variable, ‘currentView‘, dictates which heavy component is mounted into the main viewing area. The available views are:

- `import`: Mounts the `TimetableImporter`.
- `generator`: Mounts the multi-step `GeneratorView` wizard for automated timetable creation.
- `editor`: Mounts the interactive `EditorView` for drag-and-drop timetable refinement with LNS auto-fix.
- `student`: Mounts the `TimetableViewer` (All Students administrative view).
- `prof`: Mounts the `ProfessorTimetableViewer`.
- `room`: Mounts the `RoomTimetableViewer`.
- `master-room`: Mounts the campus-wide `MasterRoomViewer`.
- `free-room`: Mounts the `FreeRoomViewer`.
- `report-card`: Mounts the `ReportCardView`.

This architecture ensures that only the data required for the active view is fetched from the database, minimizing unnecessary network traffic. Custom React hooks, notably `useGeneratorData` and `useEditorData`, manage the fetching and caching of complex relational data required by these components, abstracting the Supabase query syntax away from the presentation layer.

Chapter 5

Implementation and Results

This chapter details the concrete implementation of the core algorithms and user interfaces that comprise the IIITA Timetabling System. It highlights the technical solutions developed to address the challenges outlined in previous chapters, specifically focusing on dynamic rendering, real-time conflict detection, personalized user views, the (1+1)-ES automated timetable solver, the interactive drag-and-drop Timetable Editor with LNS auto-fix, and the multi-step Generator Wizard. Finally, it discusses the results of these implementations through the lens of the final user interface.

5.1 Dynamic Time Grid Generation

A significant challenge in displaying timetables is the variability of time slots. Hardcoding a static grid (e.g., exactly ten 1-hour slots from 9:00 AM to 7:00 PM) results in wasted visual space when classes end early, and fails to accommodate non-standard time ranges (such as a 1.5-hour class).

To solve this, all viewer components in the system implement a **Dynamic Time Column Generator algorithm**. When a timetable is selected, the system fetches all associated `timetable_slots`. The algorithm then executes the following steps:

1. Extracts all unique `start_time` and `end_time` strings from the raw slot data.

2. Injects hardcoded institutional break times: `10:50` to `11:00` (Morning Break) and `13:00` to `14:30` (Lunch Break).
3. Sorts all extracted timestamps chronologically to create an array of discrete time boundaries.
4. Iterates through adjacent pairs of boundaries to generate standard columns.
5. Filters out any generated columns that fall entirely within the lunch or break periods, or that contain absolutely zero scheduled classes.

The result is a highly responsive grid that only displays columns for active periods, maximizing screen real estate and improving readability.

5.2 Multi-Dimensional Clash Detection Algorithm

One of the most critical administrative features is the automated detection of scheduling conflicts. The 'TimetableViewer' component implements a robust $O(N^2)$ algorithm that compares every slot against every other slot within the selected timetable to detect overlaps.

An overlap is mathematically defined as occurring when:

$$(SlotA.start < SlotB.end) \land (SlotA.end > SlotB.start) \tag{5.1}$$

When an overlap is detected, the algorithm categorizes the clash into one of four distinct types as summarized in Table 5.1.

These clashes are immediately surfaced to the administrator in a dedicated visual panel, complete with distinct iconography (e.g., a Map Pin for Room clashes, a User icon for Professor clashes), allowing for rapid identification and remediation.

Table 5.1: Types of Scheduling Clashes Detected

Clash Type	Logical Condition and Description
Room Clash	$SlotA.room == SlotB.room$ Two different classes are assigned to the exact same physical space simultaneously.
Professor Clash	$SlotA.professor == SlotB.professor$ A single faculty member is booked to teach two different classes at the same time.
Section Clash	$SlotA.section == SlotB.section \land SlotA.subject \neq SlotB.subject$ A specific student cohort is expected to attend two distinct subjects simultaneously.
WMC-Section	Specialized institutional rule where generic weekend/WMC sessions overlap with standard section sessions.

5.3 Inverted Logic: The Free Room Finder

Traditional scheduling systems focus on when rooms are occupied. However, students and staff frequently need to know when rooms are *available*. The 'FreeRoomViewer' component achieves this through an inverted logic algorithm.

Unlike the dynamic grid, the Free Room Finder establishes a hardcoded temporal matrix (e.g., 8:50 AM to 6:30 PM in discrete blocks).

The algorithm executes as follows:

1. The system first queries the database for **all** existing rooms.
2. It then queries the `timetable_slots` to retrieve all currently active sessions.
3. For every cell in the temporal matrix (Day × Time), it instantiates the complete set of all campus rooms.
4. It iterates through the active sessions. If a session falls within the current cell's Day and Time, that session's `room_id` is removed from the set of available rooms.

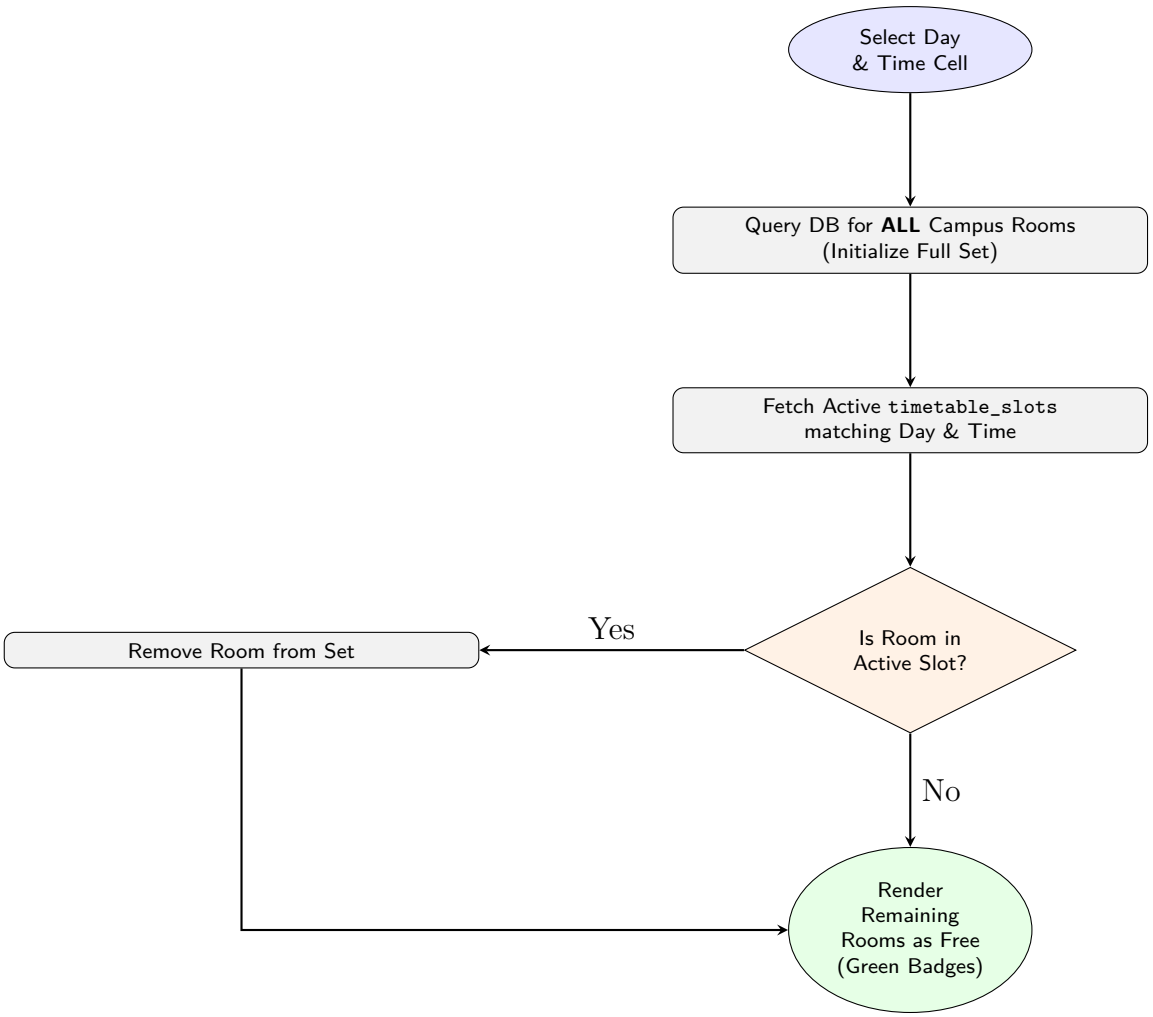

FIGURE 5.1: Inverted Logic Flowchart for the Free Room Finder Algorithm

The remaining rooms in the set are then rendered in the UI as green badges, instantly showing users where they can find empty spaces across the campus.

5.4 React Component Hierarchy

To maintain a scalable and manageable codebase, the frontend relies on a strict component tree hierarchy. Figure 5.2 illustrates the relationship between the global layout and the localized, role-based timetable viewers.

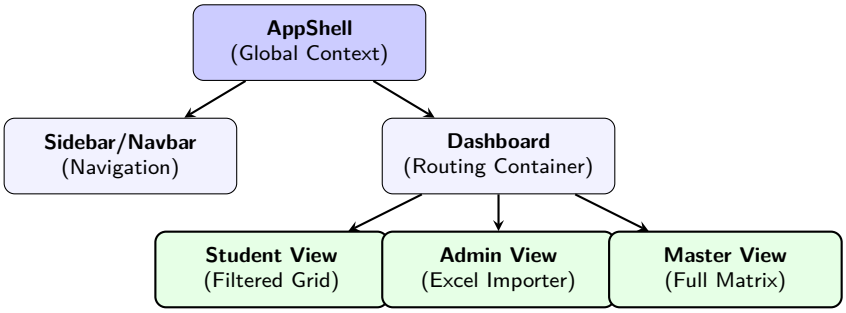

FIGURE 5.2: Hierarchical Structure of the Core React Components

5.5 Role-Based Viewer Implementations

The system provides several specialized views tailored to different stakeholders, summarized in Table 5.2.

TABLE 5.2: Role-Based Component Mapping and Data Scope

Stakeholder Role	Component View	Primary Filter Parameter
Student	`StudentCustomTimetable`	Semester & Specific Section
Professor	`ProfessorTimetableViewer`	Faculty Email/ID
Department Admin	`RoomTimetableViewer`	Specific Room Name
Campus Admin	`MasterRoomViewer`	Building Prefix (e.g., "CC-3")
Facility Staff	`FreeRoomViewer`	Temporal Matrix (Empty Rooms)

5.5.1 Student Custom Timetable

The student view is designed for maximum personalization. Given the complexity of modern curricula featuring numerous electives and minor tracks, a master timetable is overwhelming. The ‘StudentCustomTimetable‘ allows students to select their semester and specific section.

Crucially, it implements a **Smart Merging algorithm**. Often, lectures are held for "All Sections" simultaneously, while practical laboratory sessions are split by specific section (e.g., "Sec A"). The merging algorithm intelligently filters the master data, pulling in slots explicitly assigned to the student's selected section *and* slots assigned to the generic "All" section, combining them into a single, cohesive personal schedule.

Students can further refine this by toggling specific elective subjects on or off using a sidebar checklist. These preferences are persisted in the browser's ‘localStorage‘, ensuring their custom view is maintained across sessions without requiring dedicated student user accounts.

5.5.2 Master Occupancy Viewer

For campus administrators, the ‘MasterRoomViewer‘ provides a macro-level perspective. It displays a dense grid where rows represent physical rooms rather than student groups. Because an institution may have hundreds of rooms, this view implements a building-level filter. The system extracts the building prefix from the room name (e.g., identifying "CC-3" from "CC-3-1001") and allows the administrator to filter the grid, rendering a manageable overview of specific campus zones.

5.6 Export Mechanisms and Results

To bridge the gap between the digital system and offline requirements, the platform implements robust export mechanisms.

5.6.1 Client-Side PDF Generation

Users can generate PDFs of their current view. This relies on the ‘html-to-image‘ library to capture the DOM state as a PNG, which is then embedded into a ‘jsPDF‘ document. A significant technical hurdle overcome during implementation was the handling of sticky CSS headers; the capture logic temporarily disables ‘position: sticky‘ properties on the DOM elements right before capture to ensure the entire scrollable area is rendered to the image buffer accurately.

5.6.2 Google Sheets API Export

The ‘RoomTimetableViewer‘ and ‘FreeRoomViewer‘ support bulk export directly to Google Sheets. Using the Google Identity Services client, the React application constructs a multi-layered batch update request. This request not only writes the raw string data into the cells but explicitly formats the workbook—creating separate worksheets for different rooms, freezing header rows, applying pink and gray background colors to headers, and drawing solid borders—resulting in a highly professional, ready-to-share document directly in the administrator’s Google Drive.

5.6.3 User Interface Evaluation

The resulting application is a highly responsive, modern interface. By utilizing Tailwind CSS, the application maintains a clean, minimalist aesthetic with clear color coding (e.g., orange

for practicals, green for tutorials). The elimination of page reloads between views, combined with the instantaneous clash detection, represents a monumental upgrade in usability and efficiency compared to traditional spreadsheet workflows.

5.7 The (1+1)-ES Timetable Solver

The automated timetable generator is a central contribution of this project. It implements a (1+1) Evolution Strategy (ES) [**rechenberg1973evolution**, **schwefel1995evolution**] to search for conflict-free schedules that satisfy all institutional constraints. This section formalizes the problem representation, the fitness function, the mutation operators, and the adaptive control mechanisms.

5.7.1 Chromosome Representation

The solver models a complete timetable as a **Solution**, which is an ordered array of **Gene** objects. Each Gene g_i encodes the scheduling decision for the i-th class session and is defined as:

$$g_i = (\text{day}_i, \text{startBucket}_i, \text{roomIndex}_i) \tag{5.2}$$

where:

- $\text{day}_i \in \{1, 2, 3, 4, 5\}$ corresponds to Monday through Friday.
- $\text{startBucket}_i \in \{1, 2, \ldots, 8\}$ is the starting time slot (each slot represents one hour, from 08:50 to 18:30).
- roomIndex_i is the index into the available room array.

Each class session is represented by a `ClassSession` object containing immutable metadata: the subject identifier, student group, assigned professor, session duration, slot type (Lecture, Tutorial, or Practical), home room index, elective basket membership, and a locked flag indicating whether the session belongs to a previously published timetable.

5.7.2 Session Expansion: The 2+1 Lecture Format

Before solving, the `dataPrep.ts` module transforms high-level subject configurations (L-T-P counts) into individual `ClassSession` entries. A key institutional rule is the **2+1 lecture format**: for core subjects with $L \geq 2$ lectures per week, the system creates:

- Exactly **one** double-lecture block of duration 2 hours (`lecturePairIndex = 0`).
- The remaining $L - 2$ lectures as individual 1-hour sessions (`lecturePairIndex = -1`).

This is enforced as a hard constraint: the double-lecture and all single-lectures for the same subject and group must be placed on *different* days.

5.7.3 Fitness Function

The quality of a candidate solution S is evaluated by a composite fitness function:

$$f(S) = H(S) \cdot W_{\text{hard}} + G(S) \cdot W_{\text{gap}} + R(S) \cdot W_{\text{room}} + P(S) \cdot W_{\text{prof}} \tag{5.3}$$

where:

- $H(S)$ is the total number of hard constraint violations.
- $G(S)$ is the student schedule gap penalty (soft).
- $R(S)$ is the room utilization penalty (soft).

- $P(S)$ is the professor same-half penalty (soft).
- $W_{\text{hard}} = 1000$, $W_{\text{gap}} = 1.0$, $W_{\text{room}} = 0.8$, $W_{\text{prof}} = 5.0$ are configurable weights.

The large hard penalty weight ($W_{\text{hard}} = 1000$) ensures that the solver prioritizes eliminating all hard violations before optimizing soft constraints. A feasible solution has $H(S) = 0$.

5.7.4 Hard Constraints

The system enforces ten hard constraints, summarized in Table 5.3.

TABLE 5.3: Hard Constraints Enforced by the Solver

#	Constraint Name	Description
1	Time Boundary	A session must not extend beyond slot 8 (18:30).
2	Break/Lunch Crossing	A multi-slot session must not span across a break boundary (slots 2→3 or 4→5).
3	Room Non-Overlap	At most one session may occupy a given room at any (day, slot).
4	Professor Non-Overlap	A professor can teach at most one session at any given slot. No elective exemption.
5	Group Non-Overlap	A student group can attend at most one session per slot. Elective–elective pairs and same-basket pairs are exempt.
6	Elective Basket Sync	Synced baskets (HSMC, MDM) must share the same (day, slot). No two different baskets may share a slot (global anti-clash).
7	Lab Room	Practical sessions must be assigned to rooms with type "Lab".
8	WMC-Section Overlap	A whole-batch (WMC) session cannot occupy the same slot as any section-level session.
9	Home Room	Core lectures and tutorials must be placed in their assigned home room.
10	2+1 Lecture Format	The double-lecture block and single-lectures for the same subject+group must be on different days.

5.7.5 Soft Constraints

Three soft constraints provide schedule quality optimization beyond feasibility:

1. **Student Gap Penalty $G(S)$:** For each student group on each day, the penalty counts empty slots between their first and last scheduled class. Formally, if a group occupies a set of slots $\mathcal{S}_d$ on day d, the gap is $(\max(\mathcal{S}_d) - \min(\mathcal{S}_d) + 1) - |\mathcal{S}_d|$.

2. **Room Utilization Penalty $R(S)$:** For practical and elective sessions, the solver computes the utilization ratio $u = \text{studentCount}/\text{roomCapacity}$. If $u < 0.60$ (under-utilization) or $u > 1.20$ (over-utilization), a proportional penalty is applied. The result is scaled to an integer range by multiplying by 100.

3. **Professor Same-Half Penalty $P(S)$:** For each professor on each day, a penalty of 1 is incurred if they teach in both the morning half (slots 1–4) and the afternoon half (slots 5–8). This encourages compact teaching schedules.

5.7.6 Mutation Operators

The (1+1)-ES generates offspring by applying a small number of mutations to the parent solution. Each generation, the solver randomly selects 3–8% of mutable (non-locked) sessions and applies one of five mutation operators:

1. **Day Reassignment:** Assigns a uniformly random new day $d \in \{1, \ldots, 5\}$.

2. **Gaussian Time Shift:** Perturbs the start slot by $\Delta = \lfloor \mathcal{N}(0, \sigma^2) \rceil$, clamped to valid bounds. If the new position crosses a break boundary, the slot is snapped to the nearest valid start using a precomputed cache.

3. **Type-Aware Room Reassignment:** Assigns a random room respecting the session type: Lab rooms for practicals, lecture rooms for electives, and the fixed home room for core lectures/tutorials.

4. **Swap:** Exchanges the (day, slot, room) of two same-duration sessions. Synced-basket electives are excluded from swapping to preserve group synchronization.

5. **Full Relocate:** Assigns a completely new random (day, slot, room) triple.

For synced-basket electives (HSMC, MDM), mutations to day, time, or relocation are *propagated* to all members of the sync group, ensuring they remain synchronized.

5.7.7 σ-Adaptation: The 1/5th Success Rule

The step-size parameter σ controls the magnitude of Gaussian time perturbations. The solver adapts σ using Rechenberg's 1/5th success rule [**rechenberg1973evolution**]. Every $W = 50$ generations (the adaptation window), the success rate p_s (fraction of offspring that improved fitness) is computed:

$$\sigma \leftarrow \begin{cases} \sigma \times 1.22 & \text{if } p_s > 1/5 \quad \text{(too many successes} \Rightarrow \text{increase exploration)} \\ \sigma \times 0.82 & \text{if } p_s < 1/5 \quad \text{(too few successes} \Rightarrow \text{decrease step size)} \\ \sigma & \text{otherwise} \end{cases} \tag{5.4}$$

This adaptive mechanism balances exploration (large jumps in the search space) and exploitation (fine-tuning near a local optimum).

5.7.8 Stagnation Restart

If the best fitness does not improve for a configurable number of consecutive generations (the stagnation threshold, typically 25% of total generations), the solver performs a **restart**: the current solution is discarded and a fresh random initial solution is generated. The best-ever solution is preserved across restarts via elitist tracking.

5.7.9 Locked Sessions and Cross-Semester Awareness

When generating a timetable for a specific semester, the solver can incorporate **locked sessions** from previously published timetables of other semesters. These locked sessions are appended to the session array with their `isLocked` flag set to `true`. Their Gene positions are fixed and never mutated. However, they participate fully in constraint evaluation—particularly room overlap and professor overlap checks—ensuring that the new timetable does not conflict with existing published schedules.

5.7.10 Solver Execution Flow

The solver runs asynchronously in the browser using batched `setTimeout` calls (processing 200 generations per batch) to maintain UI responsiveness. The overall flow is:

1. Generate an initial solution using round-robin day assignment and break-aware slot placement.
2. If a seed timetable is provided, override matched sessions with their existing positions.
3. Pin locked genes from published timetables.
4. For each generation: mutate the parent, evaluate the offspring, accept if $f(\text{offspring}) \leq f(\text{parent})$ (elitist selection).
5. Every W generations, adapt σ via the 1/5th rule.
6. If stagnation is detected, restart with a fresh random solution.
7. Report progress (current fitness, σ, success rate, violation breakdown) to the UI at configurable intervals.

8. Terminate on reaching the maximum generation count, achieving zero hard violations, or user cancellation.

5.8 The Interactive Timetable Editor

The `EditorView` component provides an interactive, drag-and-drop interface for manually refining timetables—whether generated by the solver or imported from Excel. This section describes its key features.

5.8.1 Drag-and-Drop Slot Rearrangement

The editor renders a dense 5-day $\times$ N-slot grid, where each scheduled session appears as a draggable card. Administrators can drag any session card from one cell to another to reassign its day and start time. The system automatically recalculates the end time based on the session's duration and validates that the new position does not fall in a lunch or break slot. All drag operations push the previous state onto a 30-level undo stack.

5.8.2 Inline Editing

Each session card exposes a hover-activated edit panel that allows inline modification of:

- **Room Assignment:** A cascading two-step selector where the administrator first selects the room type (Lecture or Lab), then picks from filtered rooms of that type.
- **Professor Assignment:** A dropdown populated from the full professor list.

5.8.3 Real-Time Conflict Visualization

The editor implements a client-side clash detection algorithm that runs on every state change. Sessions involved in any conflict are highlighted with a red background and ring border.

The algorithm checks the same constraints as the solver: room overlaps, professor overlaps, section overlaps (with elective exemptions), WMC-section conflicts, cross-basket elective clashes, and the 2+1 same-day lecture rule.

5.8.4 Feasibility Check

A dedicated "Check Feasibility" button constructs a full `SolverInput` from the current editor state, converts the slot positions into a `Solution` (Gene array), appends locked sessions from all other published timetables, and invokes the solver's `evaluate()` function. The result displays the total fitness score and the per-constraint violation breakdown, giving the administrator a precise understanding of remaining issues.

5.8.5 LNS Auto-Fix

When conflicts exist, the administrator can invoke the **Large Neighbourhood Search (LNS)** auto-fix [**shaw1998using**]. This routine:

1. Identifies all session indices involved in any hard constraint violation (the "clashing set").
2. Iterates for up to 1500 attempts. In each attempt, a random session from the clashing set is selected and a random mutation (relocate, time shift, or room change) is applied.
3. If the mutation reduces the total fitness, the improved solution is accepted and the clashing set is refreshed.
4. The process terminates early if all hard violations reach zero.

The LNS targets only the problematic sessions rather than reoptimizing the entire schedule, making it significantly faster than a full solver run while preserving the administrator's

manual adjustments to non-conflicting sessions.

5.8.6 Master View

The editor includes a "Master View" toggle that fetches and overlays all slots from other published timetables as read-only, semi-transparent cards in the grid. This provides cross-semester situational awareness, allowing the administrator to visually verify that no room or professor conflicts exist between the current draft and previously published schedules.

5.9 The Generator Wizard

The `GeneratorView` component provides a guided, multi-step configuration workflow that prepares the solver input and launches the optimization process. The workflow consists of the following steps:

5.9.1 Step 1: Cluster Selection

The administrator selects a `semester_cluster` (e.g., "IT Dept, Sem 5, Batch 2023"). This scopes the scheduling problem by loading the associated subjects (with L-T-P configurations) and student groups (sections) from the database.

5.9.2 Step 2: Home Room Mapping

Each student group (section) is mapped to a home room. The system implements an automatic **floor-based home room policy**: rooms follow the naming pattern `CC-3-5`xyy, where x is the floor digit derived from the semester number (Semesters 1–2 $\rightarrow$ floor 0, 3–4 $\rightarrow$ floor 1, 5–6 $\rightarrow$ floor 2) and yy is the section suffix (A $\rightarrow$ 06, B $\rightarrow$ 07, C $\rightarrow$ 54, D $\rightarrow$ 55). This policy is applied automatically on cluster selection and can be overridden manually.

5.9.3 Step 3: Professor Assignment Matrix

An interactive matrix presents subjects as rows and student groups as columns. For each cell, the administrator selects a professor from a dropdown filtered by the `professor_expertise` table. Each subject supports four scheduling modes:

- **Sections:** Assigns individually to each section (A, B, C, D).
- **All (WMC):** Assigns to the whole-batch "WMC" group.
- **IT-BI:** Assigns only to the IT-BI interdisciplinary group.
- **Disabled:** Excludes the subject from scheduling entirely.

An "Auto-Fill" button populates all unset cells using round-robin professor assignment from the expertise pool.

5.9.4 Step 4: Seed and Lock Configuration

Before launching the solver, the administrator may optionally:

- Select a **seed timetable** to warm-start the solver from an existing schedule rather than a random initial solution.
- Select one or more **locked timetables** whose slots are injected as immutable constraints into the solver, enabling cross-semester conflict avoidance.

5.9.5 Step 5: Solver Execution and Results

Upon clicking "Generate Timetable," the system constructs the `SolverInput`, launches the (1+1)-ES solver, and displays a real-time progress card showing: current generation, best fitness, σ, success rate, and the violation breakdown. Upon completion, a results panel displays the final fitness and provides options to:

- Save the generated timetable to the database as a new draft.
- Transition directly to the `EditorView` for manual refinement.
- Download a comprehensive diagnostic log file.

Chapter 6

Conclusion and Future Work

The administration of university timetables is a complex logistical challenge that directly impacts the daily lives of students, faculty, and administrative staff. This thesis detailed the design, architecture, and implementation of the IIITA Timetabling System, a modern web-based platform built to digitize, analyze, generate, edit, and distribute academic schedules efficiently.

6.1 Summary of Contributions

The primary contribution of this project is the successful transition from static, error-prone spreadsheet management to a dynamic, intelligent, cloud-synchronized platform capable of both automated schedule generation and interactive refinement.

Specific achievements include:

- **Heuristic Ingestion Engine:** The development of a robust Excel parsing module capable of interpreting human-formatted spreadsheets. By utilizing heuristic rules to detect headers, extract metadata, and resolve cell merges, the system drastically reduces the friction of onboarding legacy data into a structured PostgreSQL database.

- **Real-Time Conflict Resolution:** The implementation of a multi-dimensional $O(N^2)$ clash detection algorithm that automatically surfaces Room, Professor, Section, and WMC-Section overlaps. This feature alone significantly reduces the administrative burden of manually verifying schedule integrity.

- **(1+1)-ES Automated Solver:** The implementation of a fully functional timetable generator using a (1+1) Evolution Strategy algorithm. The solver evaluates candidate schedules against a comprehensive fitness function comprising ten hard constraints (room, professor, and group non-overlap; break/lunch boundary enforcement; lab-room assignment; elective basket synchronization; WMC-section isolation; home-room binding; the 2+1 lecture format; and time boundary compliance) and three soft constraints (student schedule gaps, room utilization efficiency, and professor same-half-day avoidance). The algorithm employs Rechenberg's 1/5th success rule for adaptive step-size control and a stagnation restart mechanism.

- **Interactive Timetable Editor:** The development of a drag-and-drop editing interface with real-time conflict visualization, inline room and professor editing, a 30-level undo stack, and a Large Neighbourhood Search (LNS) auto-fix routine that targets clashing sessions. The editor's Master View provides cross-semester constraint awareness by overlaying slots from all published timetables.

- **Multi-Step Generator Wizard:** The creation of a guided configuration workflow enabling administrators to select semester clusters, auto-map home rooms using a floor-based naming policy, assign professors via an interactive matrix with expertise-based filtering, and configure seed and locked timetables before launching the solver.

- **Cross-Semester Constraint Awareness:** The implementation of locked sessions from published timetables, enabling the solver and editor to detect and avoid room

and professor conflicts across independently scheduled semesters.

- **Stakeholder-Specific Interfaces:** The creation of dynamic, role-based visualization components. The personalized "Student Custom Timetable" with localized preference persistence, the dedicated "Professor View," and the macro-level "Master Occupancy" and "Free Room Finder" views ensure that every user receives actionable data without cognitive overload.
- **Modern Technical Architecture:** The deployment of a high-performance frontend utilizing React 19, TypeScript, and Vite, paired with a serverless Supabase backend. This stack ensures the application is highly responsive, strictly typed, and easily maintainable.
- **Professional Export Capabilities:** The integration of client-side PDF generation and automated Google Sheets API formatting, bridging the gap between dynamic web views and the necessity for static, offline documentation.

6.2 Current Limitations and Proposed Enhancements

While the current system represents a massive upgrade over traditional methods, several limitations define the boundaries of the current implementation. These limitations directly inform the proposed future work, as summarized in Table 6.1.

6.3 Concluding Remarks

In conclusion, the IIITA Timetabling System successfully modernizes a critical institutional workflow. By combining heuristic data extraction with a robust relational database, a (1+1) Evolution Strategy solver, an interactive drag-and-drop editor with LNS-based auto-fix, and

Table 6.1: System Limitations and Future Enhancements

Domain	Current Limitation	Proposed Enhancement
Data Ingestion	Heavily reliant on hardcoded regular expressions to parse II-ITA's specific Excel format.	Implementation of a dynamic mapping UI allowing users to visually map Excel columns to DB fields.
Solver Performance	Single-threaded (1+1)-ES running on the browser's main thread via batched `setTimeout`. Sufficient for current problem sizes but may become a bottleneck for larger institutions.	Migration to Web Workers for parallel solver execution, or a server-side solver for larger problem instances. Exploration of hybrid meta-heuristics (e.g., memetic algorithms combining ES with local search).
Solver Optimality	The Evolution Strategy is a meta-heuristic; solutions are feasible but not guaranteed to be globally optimal with respect to soft constraints.	Investigation of multi-objective optimization (Pareto-optimal fronts) and hybridization with constraint programming (CP) solvers for provable optimality on subproblems.
State Management	Relies on localized React state and custom hooks, potentially causing prop-drilling in deep components.	Adoption of a global state manager (e.g., Zustand or Redux) to optimize data sharing across views.
Notifications	Users must actively check the web portal for schedule updates.	Development of a Progressive Web App (PWA) with Push Notifications via Firebase Cloud Messaging.
Analytics	Basic occupancy visualization without long-term historical analysis.	Dedicated BI Dashboard to track room utilization metrics across multiple academic years.

a lightning-fast React frontend, the system provides unparalleled visibility, automation, and integrity to academic scheduling. The system demonstrates that a browser-based, pure-TypeScript solver can reliably generate feasible, high-quality timetables for a real-world university setting, while the interactive editor empowers administrators to refine solutions with full constraint awareness. It stands as a testament to the power of modern web technologies and evolutionary computation in solving complex, real-world logistical challenges.

Appendix A

Core Algorithm Snippets

This appendix provides crucial code excerpts from the React 19 codebase, demonstrating the practical implementation of the theoretical concepts discussed in the main chapters.

A.1 Dynamic Grid Generation Logic

The following TypeScript snippet from the `TimetableViewer` component demonstrates how the system extracts unique time boundaries to dynamically generate responsive column headers, preventing the display of empty, unused time blocks.

Listing A.1: Dynamic Time Column Extraction Algorithm

```
// Extract all unique start and end times from the active slots
const allTimes = useMemo(() => {
  const times = new Set<string>();

  // Inject hardcoded institutional break times
  times.add("10:50");
  times.add("11:00");
  times.add("13:00");
  times.add("14:30");

  // Extract times from all valid timetable slots
  slots.forEach(slot => {
    if (slot.start_time && slot.end_time) {
      times.add(slot.start_time.substring(0, 5));
      times.add(slot.end_time.substring(0, 5));
    }
  });

  // Sort chronologically and format as standard columns
  return Array.from(times).sort().reduce((acc, curr, index, arr) => {
    if (index < arr.length - 1) {
      acc.push(`${curr}-${arr[index + 1]}`);
    }
    return acc;
  }, [] as string[]);
}, [slots]);
```

A.2 Real-Time Clash Detection Filter

This snippet illustrates the $O(N^2)$ comparison logic used to immediately identify Room, Professor, and Section overlaps within the currently selected semester block.

Listing A.2: Overlap Detection and Categorization Logic

```
// O(N^2) comparison to find all overlapping events
const newClashes = filteredSlots.filter((slot, i) => {
  return filteredSlots.some((otherSlot, j) => {
    // Skip self-comparison
    if (i === j) return false;
    // Must be on the same day to clash
    if (slot.day_of_week !== otherSlot.day_of_week) return false;

    // Check for temporal intersection
    const isTimeOverlap = slot.start_time < otherSlot.end_time &&
                          slot.end_time > otherSlot.start_time;

    if (isTimeOverlap) {
      // 1. Room Clash
      if (slot.rooms?.name === otherSlot.rooms?.name) return true;
      // 2. Professor Clash
      if (slot.professors?.name === otherSlot.professors?.name) return
         true;
      // 3. Section Clash (Same section, different subject)
      if (slot.student_groups?.name === otherSlot.student_groups?.name &&
          slot.subjects?.id !== otherSlot.subjects?.id) {
          return true;
      }
    }
    return false;
  });
});
```

A.3 Solver Fitness Evaluation Function

The following excerpt from `constraints.ts` demonstrates the combined fitness evaluation that scores a candidate timetable against all ten hard constraints and three soft constraints.

Listing A.3: Combined Fitness Evaluation (constraints.ts)

```
export function evaluate(input: SolverInput, solution: Solution):
   FitnessResult {
    const { sessions, rooms, numDays, config } = input;

    // Count all hard constraint violations
    const timeBoundary = countTimeBoundaryViolations(sessions, solution);
    const breakCrossing = countBreakViolations(sessions, solution);
    const roomOverlap = countRoomOverlaps(sessions, solution, rooms);
    const professorOverlap = countProfessorOverlaps(sessions, solution);
    const groupOverlap = countGroupOverlaps(sessions, solution);
    const electiveSync = countElectiveSyncViolations(sessions, solution);
    const labRoom = countLabRoomViolations(sessions, solution, rooms);
    const wmcSectionOverlap = countWMCSectionOverlaps(sessions, solution);
    const homeRoom = countHomeRoomViolations(sessions, solution);
    const twoOneLecture = countTwoOneLectureViolations(sessions, solution)
       ;

    const hardViolations = timeBoundary + breakCrossing + roomOverlap +
        professorOverlap + groupOverlap + electiveSync + labRoom +
        wmcSectionOverlap + homeRoom + twoOneLecture;

    // Compute soft constraint penalties
```

```
    const gapPenalty = computeGapPenalty(sessions, solution, numDays);
    const roomUtilizationPenalty = computeRoomUtilizationPenalty(
        sessions, solution, rooms,
        config.roomUtilizationThreshold,
        config.roomOverutilizationThreshold,
    );
    const professorSameHalfPenalty = computeProfessorSameHalfPenalty(
        sessions, solution, numDays
    );

    // Weighted composite fitness
    const total = hardViolations * config.hardPenalty
        + gapPenalty * config.gapWeight
        + roomUtilizationPenalty * config.roomUtilizationWeight
        + professorSameHalfPenalty * config.professorSameHalfWeight;

    return { total, hardViolations, gapPenalty,
        roomUtilizationPenalty, professorSameHalfPenalty,
        violationBreakdown: { timeBoundary, breakCrossing,
            roomOverlap, professorOverlap, groupOverlap,
            electiveSync, labRoom, wmcSectionOverlap,
            homeRoom, twoOneLecture },
    };
}
```

A.4 (1+1)-ES Solver Main Loop

The core optimization loop from `solver.ts`, showing elitist selection, σ-adaptation via the 1/5th success rule, and stagnation restart.

Listing A.4: (1+1)-ES Main Loop (solver.ts)

```
export async function runSolver(
    input: SolverInput,
    onProgress?: (p: SolverProgress) => void,
    cancelToken?: { cancelled: boolean },
    seedSolution?: Solution,
    lockedGenes?: Gene[],
): Promise<SolverResult> {
    const { config } = input;
    let parent = seedSolution
        ? seedSolution.map(g => ({ ...g }))
        : generateInitialSolution(input);

    // Pin locked genes from published timetables
    if (lockedGenes) {
        const offset = input.sessions.length - lockedGenes.length;
        for (let i = 0; i < lockedGenes.length; i++) {
            parent[offset + i] = { ...lockedGenes[i] };
        }
    }

    let bestFitness = evaluate(input, parent);
    let bestSolution = parent.map(g => ({ ...g }));
    let sigma = config.initialSigma;
    let successes = 0;

    for (let gen = 1; gen <= config.maxGenerations; gen++) {
```

```
        if (cancelToken?.cancelled) break;

        const offspring = mutate(parent, input, sigma);
        // Re-pin locked genes (mutations must not move them)
        if (lockedGenes) { /* ... pin logic ... */ }
        const offFitness = evaluate(input, offspring);

        // Elitist selection: accept if offspring is <= parent
        if (offFitness.total <= bestFitness.total) {
            parent = offspring;
            bestFitness = offFitness;
            bestSolution = offspring.map(g => ({ ...g }));
            successes++;
        }

        // 1/5th rule sigma adaptation every W generations
        if (gen % config.adaptationWindow === 0) {
            const rate = successes / config.adaptationWindow;
            if (rate > 0.2) sigma *= config.sigmaIncrease;
            else if (rate < 0.2) sigma *= config.sigmaDecrease;
            successes = 0;
        }

        // Stagnation restart
        if (stagnationCounter > stagnationThreshold) {
            parent = generateInitialSolution(input);
            // Re-pin locked genes ...
        }

        // Early termination if feasible
        if (bestFitness.hardViolations === 0) break;
    }
    return { solution: bestSolution, fitness: bestFitness, ... };
}
```

A.5 LNS Auto-Fix for the Editor

The Large Neighbourhood Search routine from `localSearch.ts`, which identifies all clashing sessions and iteratively repairs them.

Listing A.5: Full LNS Destroy-and-Repair (localSearch.ts)

```
export function runFullLNS(
    input: SolverInput,
    solution: Solution,
    maxAttempts: number = 1000,
): { solution: Solution; fitness: FitnessResult } | null {
    const baseFitness = evaluate(input, solution);
    if (baseFitness.hardViolations === 0) {
        return { solution, fitness: baseFitness };
    }

    // Find all session indices involved in violations
    let clashingIndices = findClashingIndices(input, solution);
    let targets = Array.from(clashingIndices);
    let bestSolution = solution.map(g => ({ ...g }));
    let bestFitness = baseFitness;
```

```
    for (let attempt = 0; attempt < maxAttempts; attempt++) {
        const candidate = bestSolution.map(g => ({ ...g }));
        // Pick a random clashing session
        const idx = targets[Math.floor(Math.random() * targets.length)];
        const session = input.sessions[idx];

        // Apply random mutation: relocate (50%), time shift (30%), room
           (20%)
        const strategy = Math.random();
        if (strategy < 0.5)
            candidate[idx] = mutateRelocate(session, input.rooms);
        else if (strategy < 0.8)
            candidate[idx] = { ...candidate[idx],
                ...mutateTime(candidate[idx], session, 3.0) };
        else
            candidate[idx] = { ...candidate[idx],
                roomIndex: mutateRoom(session, input.rooms).roomIndex };

        const candidateFitness = evaluate(input, candidate);
        if (candidateFitness.total < bestFitness.total) {
            bestSolution = candidate;
            bestFitness = candidateFitness;
            if (bestFitness.hardViolations === 0) break;
            // Refresh targets after improvement
            clashingIndices = findClashingIndices(input, bestSolution);
            targets = Array.from(clashingIndices);
        }
    }
    return bestFitness.total < baseFitness.total
        ? { solution: bestSolution, fitness: bestFitness } : null;
}
```